

PROJECT REPORT OF INDUSTRY ORIENTED HANDS ON EXPERIENCE (IOHE)

ON

RoamNest

submitted in partial fulfilment of the requirements for the award of degree of

BACHELOR OF ENGINEERING

In

COMPUTER SCIENCE AND ENGINEERING

**Submitted by:
Partha Patim Tamuly**

2210992031

Mohit K Gupta

2210991932

**Supervised By:
Dr. Preeti Saini**

Mentor

Assistant Professor

Chitkara University, Rapura



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CHITKARA UNIVERSITY INSTITUTE OF ENGINEERING AND TECHNOLOGY

CHITKARA UNIVERSITY, PUNJAB, INDIA

CONTENTS

S.No.	Title	Page No.
	Acknowledgement	iv
	Abstract / Keywords	1
1	Introduction	2-3
2	Application Flow & Use Cases	4-5
3	Software & Hardware Requirements	6
4	System Design	7-8
5	Technology Stack	8-9
6	Database Design & Implementation	10
7	Program Structure & GUI	11
8	Server-Side Logic (Code Snippets)	12-17
9	Testing & Validation	18
10	Security & Performance	19
11	Limitations	20
12	Challenges Faced	21
13	Future Scope	22
14	Technology Stack	23
15	Conclusion	24
16	References	25

DECLARATION

I hereby certify that the work which is being presented in the project report entitled “**RoamNest**” in partial fulfilment of requirement for the award of the degree of Bachelor of Engineering (Computer Science and Engineering) submitted in the department of Computer Science and Engineering at Chitkara University Institute of Engineering and Technology, Chitkara University, Punjab, India, is an authentic record of my own work carried out under the supervision of **Dr. Preeti Saini**. The matter presented in this project report has not been submitted in any other university/institute for the award of any degree.

Place: Rajpura

Partha Pratim Tamuly

Mohit Kumar Gupta

Date: 1/5/2026

2210992031

2210991932

This is to certify that the above statement made by the candidate is correct to the best of my knowledge and belief.

Dr. Preeti Saini

Asstant Professor,CUIET

Department of Computer Science and Engineering

Chitkara University Institute of Engineering and Technology,

Chitkara University, Punjab, India

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to **Dr.Preeti Saini**, my project supervisor, for her continuous guidance, valuable feedback, and encouragement throughout the course of this Co-Op Industry Project. Her insights into system design and secure software development were instrumental in shaping this project. I would also like to thank the Department of Computer Science and Engineering, Chitkara University, for providing the academic environment, resources, and opportunity to work on an industry-oriented project. Finally, I am grateful to my peers and family for their support and motivation during the development of this project

1. Abstract / Keywords

The rise of the "sharing economy" (e.g., Airbnb, Vrbo, Booking.com) has democratized travel, allowing individuals to rent out their homes to strangers at scale. However, this peer-to-peer model has introduced a significant, heavily researched social challenge: discrimination. Unlike traditional hotels, which are regulated by strict compliance frameworks and standard operating procedures, individual hosts on sharing economy platforms often unconsciously (or consciously) bias their acceptance decisions based on a guest's race, gender, name origin, or appearance.

Research consistently indicates that guests with distinctively "ethnic" names or profile photos belonging to minority groups face significantly higher rejection rates compared to others. Current platforms attempt to solve this issue with "Instant Booking" features, yet the default workflow still exposes personal identity markers before a transaction is confirmed. This fundamental design flaw allows human bias to creep into the "Accept/Reject" decision loop.

RoamNest aims to develop a rental platform architecture designed to structurally eliminate entry barriers caused by host bias. The primary objective is to re-engineer the booking workflow to prioritize merit—such as payment ability, platform behavior, and verification history—over identity, aiming to create a truly neutral marketplace through a newly conceptualized "Blind Booking" workflow.

Keywords: *RoamNest, Airbnb Clone, Spring Boot, React.js, PostgreSQL, Blind Booking, Anti-Discrimination, Role-Based Access Control (RBAC), Progressive Disclosure, Web Application, REST API, JSON Web Tokens (JWT).*

2. Introduction

2.1 Background

The sharing economy relies heavily on trust between peers. To build this trust, platforms historically mandated the sharing of personal information—names, profile pictures, and biographies—before a transaction took place. However, numerous academic studies, most notably from Harvard Business School, have demonstrated that displaying this information before a host approves a booking leads to widespread discrimination. The design flaw allows bias to dictate economic opportunities. When personal identity markers are exposed before a transaction is contractually confirmed, the subjective evaluation of a guest overshadows the objective availability of a property.

2.2 Objective of the Project

RoamNest proposes a robust software architecture designed to structurally eliminate entry barriers caused by host bias. The core objective is to enforce "Privacy-by-Design". By prioritizing objective metrics (e.g., identity verification status, historical reviews, ability to pay) over subjective markers (profile photo, name), the application aims to level the playing field for all users

2.3 Proposed Solution / System Overview

- **Design a "Blind Booking" Workflow:** Implement a progressive disclosure software system. In this system, sensitive guest attributes (Name, Profile Photo, Gender, Ethnicity markers) are heavily obfuscated from the host until the booking is contractually accepted and finalized.
- **Maintain Trust Without Bias:** Prove that safety and anonymity can coexist. Instead of relying on a profile picture to gauge safety, the platform relies on systemic "Verification Badges" (e.g., "ID Verified", "Phone Confirmed", "Email Verified") displayed during the pre-booking phase.

2.4 Key Features or Functionality

- **Role-Based Access Control (RBAC):** Distinct interfaces and API access levels for Administrators, Property Owners, and Guests (Users).

- **Progressive Disclosure Engine:** A backend fairness engine that intercepts database queries and strips out Personally Identifiable Information (PII) before sending booking request lists to Owners.
- **Dynamic Property Listing:** Owners can list, edit, and manage multiple properties with dynamic pricing and capacity limits.
- **Interactive UI:** A highly responsive React-based Single Page Application (SPA) offering real-time filtering, wishlist saving, and smooth navigation.



3. Application Flow & Use Cases

To understand the architecture of RoamNest, it is critical to understand the primary user journeys that the system supports.

3.1 User Registration and Authentication Flow

Users land on the RoamNest application and can choose to register as either a USER (Guest) or an OWNER (Host). Upon submission, the Spring Boot backend hashes the password using BCrypt and stores the profile in PostgreSQL. Upon logging in, the backend authenticates the credentials and issues a stateless JSON Web Token (JWT) valid for 24 hours. This token dictates what UI elements the React frontend displays.

3.2 Owner Property Listing Flow

A user with the OWNER role accesses their dedicated dashboard. They can create a new property listing by providing a title, description, location, maximum guest capacity, and price per night. The data is securely transmitted via a POST request with the JWT in the Authorization header.

3.3 Guest Booking Flow

A user with the USER role browses the platform, utilizing filters (location, price, dates). Upon selecting a property, they initiate a booking request. The system verifies property availability for the selected dates and generates a PENDING booking record in the database.

3.4 The "Blind Booking" Owner Decision Flow

This is the core differentiating flow of the project.

1. The Owner receives a notification of a new booking request.
2. The Owner navigates to their OwnerBookings dashboard.
3. The backend API (/api/bookings/owner) retrieves the bookings but **deliberately maps the data to a specialized DTO (Data Transfer Object)** that hides the

guest's name and contact info. Instead of "John Doe wants to book", the API sends "Guest RN-1024 wants to book for 3 nights (ID Verified)".

4. The Owner decides strictly based on their calendar availability and the guest's verified status.
5. Once the Owner clicks "Approve", the database state changes to APPROVED, and only then does the backend progressively disclose the guest's real identity to facilitate check-in logistics.

4. Software & Hardware Requirements

4.1 Software Requirements

Component	Technology Used	Version
Runtime Environment	Node.js, Java Development Kit (JDK)	Node v18+, Java 17
Frontend Framework	React.js via Vite Build Tool	React 19, Vite 8
Backend Framework	Spring Boot	4.0.4
Database Management	PostgreSQL	15+
Security Architecture	Spring Security, jjwt-api	Latest
API Testing	Postman	Latest
Version Control	Git & GitHub	N/A
IDE	Visual Studio Code, IntelliJ IDEA	Latest

4.2 Hardware Requirements

Component	Specification
Processor	Dual-core 2 GHz or higher
RAM	Minimum 8 GB
Storage	10 GB free disk space
Network	Stable broadband internet connection
Browser	Google Chrome / Firefox / Edge (latest)

5. System Design

5.1 Architecture Overview

RoamNest is built using a modern **Client-Server Single Page Application (SPA)** architecture.

- **The Client (Frontend):** Developed entirely in React.js. It manages the user interface, client-side routing (React Router), state management, and form validations. It communicates with the backend exclusively via asynchronous HTTP requests using Axios.
- **The Server (Backend):** Developed in Spring Boot. It acts as a strict RESTful API provider. It contains no view-rendering logic (no JSP or Thymeleaf). It strictly receives JSON, processes business rules, interacts with the database, and returns JSON.
- **The Database:** A PostgreSQL relational database ensuring ACID compliance, crucial for handling concurrent booking transactions without double-booking a property.

5.2 Role-Based Access Control (RBAC) System

The system implements a rigid RBAC matrix.

- **ADMIN:** Has overarching system viewing capabilities.
- **OWNER:** Can POST to /api/properties, GET their specific booking queue, and PATCH booking statuses. Cannot make bookings.
- **USER:** Can GET properties, POST to /api/bookings, and GET their own booking history. Cannot create properties or approve bookings.

5.3 Role-Based Access Control (RBAC) System

The system architecture treats data visibility as a state-machine function.

- State PENDING: Guest Identity variables = NULL.
- State APPROVED: Guest Identity variables = POPULATED.
- State REJECTED: Guest Identity variables = NULL. By embedding this rule deeply within the backend Service layer, the system guarantees that the frontend UI cannot accidentally render biased data, as the data never reaches the client's browser in the first place.

6. Database Design & Implementation

6.1 Entity Relationship Model

The PostgreSQL database is fully normalized to Third Normal Form (3NF).

- **Users Table (user_data):** The central entity storing authentication credentials, roles, and profiles.
- **Properties Table (properties):** Contains listing details, linked via a Foreign Key to the user_data table (Owner ID).
- **Bookings Table (bookings):** The transactional junction table linking a Property ID and a Guest ID, containing date ranges, guest count, and an ENUM for status (PENDING, APPROVED, REJECTED, CANCELLED).
- **Wishlist Table (wishlists):** A many-to-many resolution table allowing a user to save multiple properties.
- **Messages Table (booking_messages):** Facilitates post-approval communication.

6.2 The User Entity Model

Below is the exact implementation of the User database model using Spring Data JPA. Note the use of `@Entity` and `@Table` to map the Java class directly to a PostgreSQL table, and the enforcement of database-level constraints (nullable = false, unique = true).

```
package com.roamnest.backend.entity;

import jakarta.persistence.*;

@Entity // Tells Spring "Make a database table out of this class"
@Table(name = "user_data") // Tells Spring to name the table "user_data"
public class User {

    @Id // Denotes the primary key
    @GeneratedValue(strategy = GenerationType.IDENTITY) // Auto-increment
    strategy
    private Long id;

    // Spring translates camelCase 'fullName' to snake_case 'full_name' in
    SQL
    private String fullName;

    // We enforce rules at the schema level to ensure data integrity
    @Column(nullable = false, unique = true)
    private String username;

    @Column(nullable = false)
    private String password;

    @Column(nullable = false)
    private String role; // Stores "ADMIN", "OWNER", or "USER"

    @Column(nullable = false)
    private boolean enabled = true;

    private String phoneNo;
    private String address;

    // Required default constructor for JPA instantiation
    public User() {}

    // Getters and Setters omitted for brevity but required for encapsulation
    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }
    public String getUsername() { return username; }
    public void setUsername(String username) { this.username = username; }
    // ...
}
```

7. Technology Stack

7.1 Frontend: React 19, Vite, React Router

- **React 19:** Utilized for building a modular, component-based user interface. Functional components and hooks (useState, useEffect, useCallback) are used extensively to manage state.
- **Vite:** Chosen over Create React App (CRA) due to its exceptionally fast Hot Module Replacement (HMR) and optimized build speeds utilizing ES modules.
- **React Router v7:** Implements client-side routing, preventing full page reloads and providing a native-app-like experience.

7.2 Backend: Spring Boot 4.0.4, Java 17

- **Spring Boot:** Drastically reduces boilerplate configuration, providing a production-ready application context out of the box.
- **Spring WebMVC:** Provides the @RestController annotations and robust request/response handling.
- **Spring Data JPA:** Acts as an Object Relational Mapper (ORM), allowing developers to interact with the database using Java methods instead of writing raw SQL strings, significantly reducing SQL injection vulnerabilities.

7.3 Security: Spring Security & JWT

- **Stateless Authentication:** JSON Web Tokens (JWT) are mathematically signed strings. When a user authenticates, a JWT is issued. For all subsequent requests, the React frontend includes this token in the Authorization: Bearer <token> header.
- **Why JWT?:** Because it removes the need for the server to store session IDs in memory, making the backend completely stateless and horizontally scalable.

8. Program Structure & GUI

8.1 Backend Directory Structure (MVC Pattern)

The Spring Boot application strictly follows the Controller-Service-Repository pattern.

- controller/: The entry point for HTTP requests. Validates incoming JSON and delegates to the Service.
- service/: The "Brain" of the application. Contains all business logic, date overlap checks, and fairness algorithms.
- dao/: Data Access Objects (Repositories). Interfaces extending JpaRepository for database communication.
- security/: Contains the custom JwtAuthenticationFilter that intercepts requests to validate tokens before they reach the controllers.

8.2 Frontend Pages

- Home.jsx / Properties.jsx: Listing of available properties for rent.
- PropertyDetail.jsx: Detailed view with booking action.
- Login.jsx & Signup.jsx: User authentication screens.
- OwnerBookings.jsx: Specialized view for Owners to approve/reject bookings without seeing guest identity markers.
- Profile.jsx & AdminDashboard.jsx: Role-specific management pages.

9. Server-Side Logic — Code Snippets

9.1 Frontend Routing & RBAC (App.jsx)

The frontend must protect routes to ensure a User cannot navigate to an Owner dashboard.

This is achieved via a Custom Route wrapper component (RequireRole).

```
import { BrowserRouter, Routes, Route, Navigate } from 'react-router-dom';
import { useState, useEffect } from 'react';
// Imports omitted for brevity...

function App() {
  const [userRole, setUserRole] = useState(null);
  const [username, setUsername] = useState(null);

  // Restore session from sessionStorage on browser refresh
  useEffect(() => {
    const raw = sessionStorage.getItem('rn_auth');
    if (raw) {
      try {
        const { role, username: user } = JSON.parse(raw);
        if (role && user) {
          setUserRole(role);
          setUsername(user);
        }
      } catch {
        sessionStorage.removeItem('rn_auth');
      }
    }
  }, []);

  // Custom component to enforce Role-Based Access Control on the client side
  const RequireRole = ({ roles, children }) => {
    if (!userRole) return <Navigate to="/" />; // Not logged in -> kick to login
    if (!roles.includes(userRole)) return <Navigate to="/home" />; // Wrong role -> kick to home
    return children; // Authorized -> render the page
  };

  return (
    <BrowserRouter>
      <Routes>
        <Route path="/" element={userRole ? <Navigate to="/home" /> : <Login />} />

        {/* USER-only Routes */}
        <Route path="/wishlists" element={
          <RequireRole roles={['USER']}><Wishlists /></RequireRole>
        } />

        {/* OWNER-only Routes */}
        <Route path="/owner/bookings" element={
```



```

        <RequireRole roles={['OWNER']}><OwnerBookings /></RequireRole>
      } />

      {/* Shared Routes */}
      <Route path="/properties" element={
        <RequireRole roles={['USER', 'OWNER']}><Properties /></RequireRole>
      } />
    </Routes>
  </BrowserRouter>
);
}
export default App;
}
}

```

9.2 Dynamic UI Filtering (Properties.jsx)

The Properties page demonstrates complex state management in React, allowing users to filter listings dynamically. When filters change, it triggers an API call via Axios .

```

import { useState, useEffect, useCallback } from 'react';

import api from '../api'; // Centralized Axios instance

export default function Properties({ role, username }) {
  const [properties, setProperties] = useState([]);
  const [loading, setLoading] = useState(false);
  const [filters, setFilters] = useState({ location: '', minPrice: '', sort:
'newest' });

  // Memoized search function to prevent infinite re-rendering loops
  const search = useCallback(() => {
    setLoading(true);
    const params = {};
    if (filters.location.trim()) params.location = filters.location.trim();
    if (filters.minPrice) params.minPrice = filters.minPrice;
    params.sort = filters.sort;

    // Asynchronous API call to Spring Boot backend
    api.get('/properties/search', { params })
      .then(response => setProperties(response.data))
      .catch(() => setProperties([]))
      .finally(() => setLoading(false));
  }, [filters]);

  // Fetch properties on initial component mount
  useEffect(() => { search(); }, []);

  return (
    <div>
      <h1>Find Your Stay</h1>
      {/* Filtering UI implementation */}

```

```

    <div>
      <input
        type="text"
        placeholder="Location"
        value={filters.location}
        onChange={e => setFilters(prev => ({ ...prev, location:
e.target.value })))}
      />
      <button onClick={search}>Search</button>
    </div>

    {/* Conditional Rendering based on loading state */}
    {loading ? (
      <p>Searching...</p>
    ) : (
      <div className="grid">
        {properties.map(p => (
          <div key={p.id} className="card">
            <h3>{p.title}</h3>
            <p>{p.location} • ₹{p.pricePerNight}/night</p>
          </div>
        ))}
      </div>
    )}
  </div>
);
}

```

9.3 Authentication & Security Configuration

The application uses Spring Security to protect endpoints globally. The security configuration disables traditional session management in favor of a stateless JWT approach. It also configures CORS to allow requests from the React frontend running on port 5173.

```

// hooks/use-fetch.js
import { useState } from "react";
import { toast } from "sonner";

const useFetch = (cb) => {
  const [data, setData] = useState(undefined);
  const [loading, setLoading] = useState(null);
  const [error, setError] = useState(null);

  const fn = async (...args) => {
    setLoading(true);
    setError(null);
    try {
      const response = await cb(...args);
      setData(response);
      setError(null);
    } catch {
      toast.error("An error occurred");
    }
  };

  return { data, loading, error, fn };
};

```

```

    } catch (error) {
        setError(error);
        toast.error(error.message);
    } finally {
        setLoading(false);
    }
};

return { data, loading, error, fn, setData };
};

export default useFetch;

```

9.4 SecurityConfig.java

```

package com.roamnest.backend.security;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.config.annotation.web.configuration.EnableWebSecurity;
import org.springframework.security.config.http.SessionCreationPolicy;
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.security.web.SecurityFilterChain;
import org.springframework.security.web.authentication.UsernamePasswordAuthenticationFilter;

@Configuration
@EnableWebSecurity
public class SecurityConfig {

    private final JwtAuthenticationFilter jwtAuthenticationFilter;

    public SecurityConfig(JwtAuthenticationFilter jwtAuthenticationFilter) {
        this.jwtAuthenticationFilter = jwtAuthenticationFilter;
    }

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
        return http
            .cors(cors -> {}) // Enable CORS
            .csrf(csrf -> csrf.disable()) // Disable CSRF as we use stateless JWTs
            .sessionManagement(session ->
                session.sessionCreationPolicy(SessionCreationPolicy.STATELESS))

```

```

        .authorizeHttpRequests(auth -> auth
            // Allow unauthenticated access to login and registration
            .requestMatchers("/api/auth/signup",
"/api/auth/login").permitAll()
            // Require authentication for all other endpoints
            .anyRequest().authenticated())
        // Inject our custom JWT filter BEFORE the standard Spring
authentication filter
        .addFilterBefore(jwtAuthenticationFilter,
UsernamePasswordAuthenticationFilter.class)
        .build();
    }

    @Bean
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder(); // Strong cryptographic hashing
    }
}

```

9.5 The Blind Booking Protocol Implementation

The progressive disclosure engine is implemented in the `BookingService.java`. When an owner fetches their booking queue, the service maps the raw database record (`OwnerBookingRecord`) to a Data Transfer Object (`OwnerBookingResponse`). During this mapping, the code explicitly checks the status of the booking. If the booking is NOT APPROVED, the guest's name, username, and contact details are set to null, ensuring the data never travels over the network to the frontend.

9.6 BookingService.java & BookingController.java

```

//BookingController.java//
/**
 * Owner-side booking queue endpoint.
 * Guest identity is never included in the response – only an opaque
guestRef.
 */
@GetMapping("/owner")
public ResponseEntity<List<OwnerBookingResponse>> listOwnerBookings (
    Authentication authentication,
    @RequestParam(required = false) String status) {
    return ResponseEntity.ok(
        bookingService.listOwnerBookings(authentication.getName(),
status)
    );
}

```

```
//BookingService.java (The fairness logic)//
```

```
public List<OwnerBookingResponse> listOwnerBookings(String username, String
status) {

    UserAccount owner = getCurrentUser(username);
    requireRole(owner, "OWNER", "Only owners can view their booking
requests");

    return bookingDao.findByOwnerIdAndStatus(owner.getId(), status)
        .stream()
        .map(this::toOwnerResponse) // Maps entity to DTO applying
privacy rules
        .toList();
}

private OwnerBookingResponse toOwnerResponse(OwnerBookingRecord rec) {
    // Create an opaque identifier to hide the guest's real identity
    String guestRef = "RN-" + rec.getId();

    // PROGRESSIVE DISCLOSURE LOGIC: Only show guest details IF approved
    boolean canSeeGuestDetails = "APPROVED".equals(rec.getStatus());

    return new OwnerBookingResponse(
        rec.getId(),
        guestRef,
        // Conditional data population based on status
        canSeeGuestDetails ? rec.getGuestUserId() : null,
        canSeeGuestDetails ? rec.getGuestFullName() : null,
        canSeeGuestDetails ? rec.getGuestUsername() : null,
        canSeeGuestDetails ? rec.getGuestPhoneNo() : null,
        canSeeGuestDetails ? rec.getGuestAddress() : null,
        rec.getPropertyId(),
        rec.getPropertyTitle(),
        rec.getPropertyLocation(),
        rec.getCheckInDate(),
        rec.getCheckOutDate(),
        rec.getGuests(),
        rec.getStatus()
    );
}
```

10. Testing & Validation

10.1 API Testing (Postman)

The backend REST APIs were rigorously tested using **Postman**. Test suites were created for different roles:

- **Authentication Flow:** Verify that sending invalid credentials returns 401 Unauthorized.
- **Role Validation:** Ensure that an endpoint annotated for ADMIN returns 403 Forbidden if a USER JWT token is provided.
- **Privacy Enforcement:** Sent a GET request to /api/bookings/owner for a pending booking and verified via raw JSON inspection that fields like guestFullName were definitively null.

10.2 Date Overlap Testing

Complex test scenarios were executed to ensure double-bookings are impossible. If Property A is booked from May 10th to May 15th, any subsequent booking request that overlaps (e.g., May 12th to May 16th) immediately triggers a 409 Conflict database exception handled gracefully by the controller.

10.2 UI & End-to-End Testing

Manual end-to-end (E2E) testing was conducted across multiple browsers (Chrome, Edge, Firefox). The responsive design was tested by simulating mobile device viewports (375px width) to ensure the CSS grid layouts for property cards collapsed correctly.

11. Security & Performance

- **Stateless Authentication:** By utilizing JWT, the server requires zero memory to track active sessions. This significantly enhances the application's ability to handle high traffic loads concurrently.
- **Password Hashing:** Passwords are never stored in plaintext. Spring Security's BCryptPasswordEncoder applies a slow hashing algorithm with a unique salt per user, rendering database dumps useless to attackers attempting dictionary attacks.
- **CORS Protection:** Cross-Origin Resource Sharing is strictly configured to only accept requests originating from the authorized frontend origin (<http://localhost:5173>), preventing malicious scripts on other domains from hijacking the API.
- **SQL Injection Prevention:** By utilizing Spring Data JPA and Hibernate under the hood, all database interactions use parameterized queries. This completely neutralizes traditional SQL injection attack vectors.
- **Efficient Filtering:** The frontend implements debouncing and memoization (`useCallback`) to prevent excessive API calls while a user is typing in the location filter search bar.

12. Limitations

1. **Scope of Anonymity:** The blind booking anonymity only covers the initial request and decision phase. Post-acceptance, identity details must be shared to facilitate actual physical check-ins and legal liability tracking.
2. **Third-Party Verification:** The concept of "Verification Badges" currently relies on internal platform trust schemas rather than integrations with formal, government-backed third-party KYC (Know Your Customer) identity providers like Onfido or Stripe Identity.
3. **Real-time Synchronization:** The messaging system and booking status updates rely on standard HTTP polling or manual page refreshes. Real-time host-to-guest chat lacks WebSocket (e.g., Socket.io) integration.
4. **Payment Processing:** The current architecture abstracts away financial transactions, focusing purely on the ethical and logistical booking logic. It lacks a real-time payment gateway integration.

13. Challenges Faced

1. Enforcing Anonymity in API Responses:

Ensuring that guest details never accidentally leaked in JSON payloads required careful DTO design. If an entity model was accidentally serialized directly to JSON instead of the DTO, it would defeat the entire purpose of the application.

2. Complex Role-Based Access Control :

Differentiating permissions securely across two entirely separate applications (React and Spring Boot) for three distinct roles without creating excessive code duplication proved challenging.

3. Database Schema Constraints:

Modeling complex relational constraints where a single Booking transaction involves a Property, an Owner (User), and a Guest (User), while ensuring the "Blind" constraint is respected at the service layer.

4. CORS and Token Management:

Handling Cross-Origin Resource Sharing (CORS) between the Vite frontend development server and the Spring Boot backend required specific configurations to ensure HTTP OPTIONS pre-flight requests succeeded alongside the Authorization header.

14. Future Scope

- **Dynamic Pricing Engine:** Implementation of an AI-based algorithm that suggests or automatically adjusts property pricing based on local demand, seasonality, and local events.
- **Automated Moderation:** Integration of a robust review moderation system utilizing Natural Language Processing (NLP) to automatically detect and flag toxic, discriminatory, or biased feedback in property reviews.
- **Real-Time WebSockets:** Upgrading the message center and booking notifications from HTTP requests to persistent WebSockets for instant, real-time communication between hosts and guests.
- **KYC Integration:** Full integration with external background check APIs to issue highly trusted verified identity badges, further removing the need for profile photos as trust proxies.
- **Mobile Application:** Expanding the platform ecosystem by developing a dedicated native mobile application for iOS and Android utilizing React Native, sharing the same RESTful Spring Boot backend.

15. Conclusion

RoamNest demonstrates a robust technical solution to a profound social issue—discrimination within the sharing economy. By implementing a "Blind Booking" Progressive Disclosure Protocol, the system successfully shields users from unconscious bias during the critical decision-making phase of renting.

Powered by a modern, high-performance stack utilizing React.js, Spring Boot, and PostgreSQL, the application enforces ethical design directly at the architectural level. RoamNest effectively proves that safety, trust, and complete anonymity can coexist, offering a scalable blueprint for the next generation of inclusive and fair peer-to-peer marketplaces.

16. References

- [1] Spring Boot Reference Documentation — <https://spring.io/projects/spring-boot>
- [2] React.js Official Documentation — <https://react.dev/>
- [3] PostgreSQL Official Documentation — <https://www.postgresql.org/docs/>
- [4] JSON Web Tokens (JWT) Introduction and Debugger — <https://jwt.io/>
- [5] "Racial Discrimination in the Sharing Economy: Evidence from a Field Experiment" — Edelman, B., Luca, M., & Svirsky, D. (American Economic Journal: Applied Economics).
- [6] Vite Next Generation Frontend Tooling — <https://vitejs.dev/>
- [7] Spring Security Reference Architecture — <https://docs.spring.io/spring-security/reference/>
- [8] React Router v7 Documentation — <https://reactrouter.com/>
- [9] "Privacy by Design" Principles — Ann Cavoukian, Ph.D.
- [10] Axios Promise based HTTP client for the browser and node.js — <https://axios-http.com/>